Prometheus in DevOps

A Comprehensive Monitoring Solution for Modern Infrastructure

Presented by:

Kashan Saeed

Ikrama Shah

Muhammad Asif

Muhammad Baqir

Waleed Kayani

Introduction to Prometheus

Open-source monitoring system with a dimensional data model

- **Time Series Database** - Stores metrics as timestamped values with labels
- **Pull-based Monitoring** - Scrapes metrics from instrumented jobs
- **PromQL** - Powerful query language for time series data
- **Alerting** - Built-in alerting with Alertmanager
- **Service Discovery** - Automatically discovers monitoring targets

Prometheus

Why Prometheus in DevOps?

Provides reliable monitoring for dynamic cloud-native environments, enabling proactive issue detection and performance optimization

Prometheus History

From a monitoring tool at SoundCloud to a CNCF graduated project

2012

Origin at SoundCloud

Created by Julius Volz and Matthias Rademacher to solve monitoring challenges in a cloud-native environment

2015

Open Source Release

Prometheus v1.0 released to the public with a pull-based monitoring model and time series database

2016

Joined CNCF

Became the second project after Kubernetes to join the Cloud Native Computing Foundation

2018

Graduated Project

Achieved graduated status from CNCF, demonstrating maturity, stability, and widespread adoption

2019

Prometheus 2.0

Major release with improved storage engine, better performance, and new query capabilities

2022

Industry Standard

Adopted by major cloud providers and became the de facto standard for cloud-native monitoring

Prometheus Architecture

Core components and their interactions in the Prometheus monitoring ecosystem

Prometheus Server

Central component that scrapes metrics, stores time series data, and evaluates alerting rules

Exporters

Collect metrics from third-party systems and expose them in Prometheus format

Pushgateway

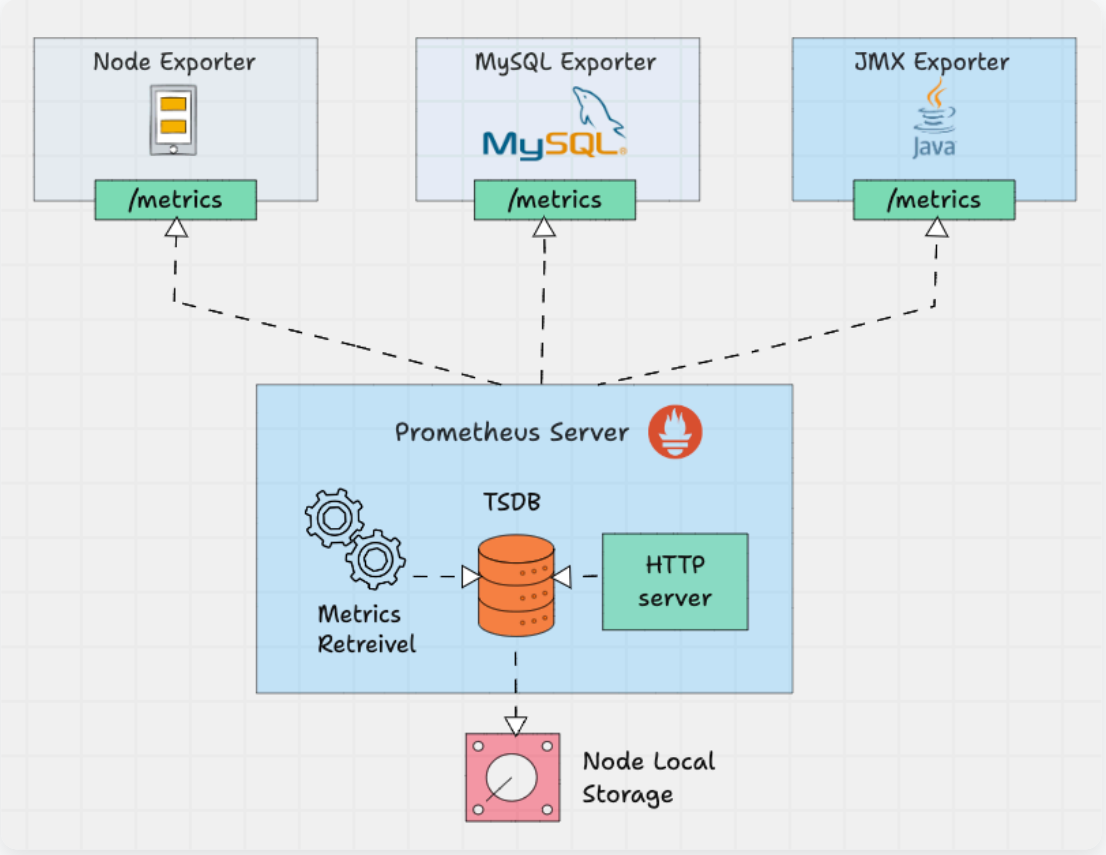
Intermediate cache for metrics from short-lived jobs that cannot be scraped

Alertmanager

Handles alert routing, grouping, inhibition, and silencing

Service Discovery

Automatically discovers monitoring targets in dynamic environments



Prometheus Data Model

Time series data with dimensional labels and multiple metric types

Metric Types

Counter

Cumulative value that only increases (e.g., requests_total)

Gauge

Value that can go up or down (e.g., memory_usage)

Histogram

Samples observations in configurable buckets (e.g., request_duration_seconds)

Summary

Similar to histogram but with configurable quantiles (e.g., rpc_duration_seconds)

Labels

Key-value pairs that add dimensions to metrics, enabling powerful filtering and aggregation

Examples

Metric with Labels

```
http_requests_total{method="GET",status="200"}
1027
```

```
http_requests_total{method="POST",status="404"} 34
```

Time Series Data

```
metric_name{label1="value1",label2="value2"} value
timestamp
```

```
node_cpu_seconds_total{cpu="0",mode="idle"}
12345.67 1667890123
```

PromQL

Powerful query language for time series data in Prometheus

Query Types

Instant Vectors

Selects a single value at a specific timestamp

Range Vectors

Selects a range of values over time

Aggregation

Combines multiple time series into one

Functions

Built-in functions for calculations

Key Features

Label-based filtering, arithmetic operations, comparison operators, and built-in aggregation functions

Query Examples

Basic Filtering

```
http_requests_total{method="GET",status="200"}
```

Rate Calculation

```
rate(http_requests_total[5m])
```

Aggregation

```
sum(http_requests_total) by(method)
```

Prediction

```
predict_linear(node_filesystem_free_bytes[1h],
3600)
```

Installation and Configuration

Step-by-step commands to install and configure Prometheus

Installation Methods

Linux

```
wget https://github.com/prometheus/prometheus/releases/download/v2.40.0/prometheus-2.40.0.linux-amd64.tar.gz
```

```
tar xvfz prometheus-2.40.0.linux-amd64.tar.gz
```

```
cd prometheus-2.40.0.linux-amd64
```

```
./prometheus --config.file=prometheus.yml
```

Docker

```
docker run -p 9090:9090 prom/prometheus
```

```
docker run -p 9090:9090 -v /path/to/prometheus.yml:/etc/prometheus/prometheus.yml prom/prometheus
```

Kubernetes

```
kubectl apply -f prometheus-config.yaml
```

```
kubectl apply -f prometheus-deployment.yaml
```

Configuration

Basic Configuration File

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s
rule_files:
  - "rules/*.yml"
scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']
```

Prometheus Exporters

Bridges between Prometheus and various systems for metrics collection

Popular Exporters

Node Exporter

Hardware and OS metrics for Linux systems

Blackbox Exporter

Probes endpoints over HTTP, HTTPS, DNS, TCP, ICMP

MySQL Exporter

Database performance metrics for MySQL

DNS Exporter

DNS server response time and query metrics

Redis Exporter

Redis server performance metrics

Kubernetes Exporter

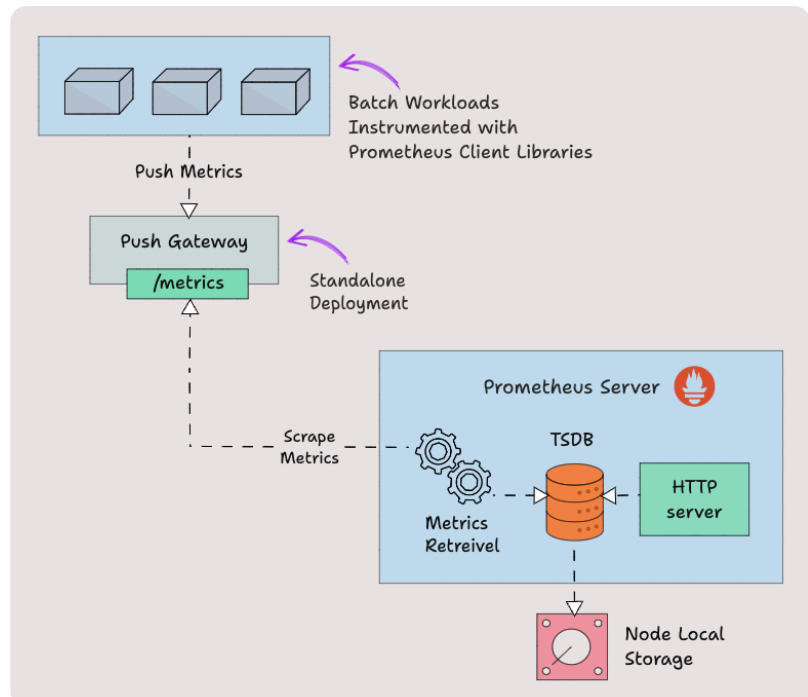
Container orchestration metrics

How Exporters Work

Exporters collect metrics from target systems and expose them in Prometheus format via HTTP endpoints

Exporter Integration

Exporter Architecture



Configuration Example

```yaml
scrape_configs:
  - job_name: 'node'
    static_configs:
      - targets: ['localhost:9100']
  - job_name: 'blackbox'
    metrics_path: /probe
    params:
      module: [http_2xx]
    static_configs:
      - targets: ['example.com']
```

Prometheus in DevOps Workflows

Integrating Prometheus across the DevOps lifecycle for continuous observability

DevOps Pipeline Integration

Development

Instrument code with Prometheus client libraries to expose custom metrics

CI/CD

Monitor build pipelines, test performance, and deployment success rates

Deployment

Track application health, resource utilization, and error rates

Operations

Continuous monitoring, alerting, and performance optimization

Key Integrations

Toolchain Compatibility

- **Kubernetes** - Native monitoring for containers
- **Grafana** - Visualization and dashboards
- **Alertmanager** - Alert routing and management
- **Thanos/Cortex** - Long-term storage solutions
- **Terraform** - Infrastructure as Code

DevOps Benefits

Prometheus enables **observability-as-code**, supporting the DevOps principle of treating monitoring as an integral part of the software development lifecycle

Alerting with Prometheus

Proactive monitoring with Alertmanager and alert rules

Alertmanager Features

Grouping

Combines related alerts into single notifications

Routing

Directs alerts to appropriate receivers

Inhibition

Suppresses certain alerts when others are firing

Silencing

Temporarily mutes specific alerts

Alert Flow

Rule Evaluation → Alert Firing → Alertmanager → Notification

Alert Rules Configuration

Alert Rule Example

```yaml
groups:
  - name: example_alerts
    rules:
      - alert: HighErrorRate
        expr: rate(http_requests_total{status=~"5.."}[5m]) > 0.05
        for: 5m
        labels:
          severity: critical
        annotations:
          summary: "High error rate detected"
```

Alertmanager Configuration

Advanced alert routing and notification configuration

Configuration Structure

Alertmanager Configuration

```
route:
  group_by: ['alertname']
  group_wait: 10s
  group_interval: 10s
  repeat_interval: 1h
  receiver: 'web.hook'
```

Notification Channels

Email

Sends alert notifications via email

Slack

Posts alerts to Slack channels

Webhooks

Sends HTTP POST requests to endpoints

PagerDuty

Integrates with PagerDuty for on-call

Visualization

Creating insightful dashboards with Grafana and Prometheus

Grafana Features

PromQL Support

Native support for Prometheus query language

Dashboard Templates

Pre-built templates for common monitoring scenarios

Visualization Types

Graphs, heatmaps, gauges, tables, and more

Alert Integration

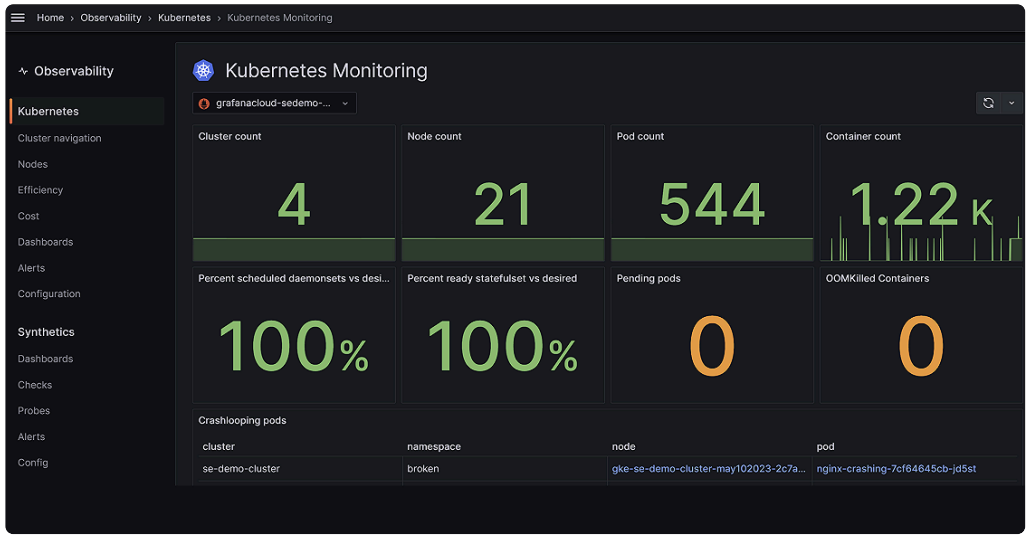
Visual alerting with Prometheus alerts

Integration Steps

1. Add Data Source
2. Write PromQL
3. Create Panels
4. Build Dashboard

Dashboard Examples

Kubernetes Monitoring Dashboard



System Performance Dashboard

Prometheus Best Practices

Tips for effective monitoring with Prometheus

Key Practices

Metric Naming

Use **snake_case** with meaningful names that reflect what is being measured

Label Usage

Add **consistent** labels for dimensions like service, instance, or environment

Cardinality

Avoid **high cardinality** labels like user IDs or timestamps

Query Optimization

Use **recording rules** for expensive queries to improve performance

Retention Strategy

Configure appropriate retention periods based on storage capacity and analysis needs. Consider long-term storage solutions for historical data

Dos and Don'ts

Do

- ✓ Use **consistent** metric names
- ✓ Monitor **business metrics**
- ✓ Set **alert thresholds** carefully
- ✓ Document **SLI/SLO** metrics

Don't

- ✗ Mix **units** in one metric
- ✗ Use **high cardinality** labels
- ✗ Store **logs** as metrics
- ✗ Create **alert fatigue**

Architecture Tips

Use federation for multi-environment monitoring and consider long-term storage solutions like Thanos or Cortex for scalability

Use Cases

Real-world applications of Prometheus monitoring

Kubernetes Monitoring

- Container resource usage
- Pod health and restarts
- Network traffic between services

Example Dashboard

Kubernetes cluster overview with node health, pod distribution, and resource utilization metrics

Microservices

- Request latency percentiles
- Error rates by service
- Request throughput

Example Dashboard

Service mesh visualization with request flow, latency heatmap, and error distribution

Infrastructure

- CPU and memory utilization
- Disk space and I/O metrics
- Temperature and hardware health

Example Dashboard

System overview with server health status, resource utilization trends, and capacity planning

Applications

- Business metrics (orders, users)
- Database query performance
- User engagement metrics

Example Dashboard

Application performance with business KPIs, user activity, and system resource correlation

Comparison with Other Monitoring Tools

Strengths and weaknesses of Prometheus compared to other monitoring solutions

Tool	Architecture	Key Features	Strengths	Weaknesses
Prometheus	Pull-based, time series DB	PromQL, Alerting, Service Discovery	Cloud-native, powerful query language, active community	Limited long-term storage, steep learning curve
Nagios	Agent-based, poll-based	Plugins, Notifications	Mature, extensive plugin ecosystem	Complex configuration, limited metrics visualization
Zabbix	Agent-based, centralized	Dashboard, Graphing	All-in-one solution, auto-discovery	Resource-intensive, complex scaling
Datadog	SaaS-based, agent-based	Integrations, APM	Easy setup, comprehensive features	Expensive, vendor lock-in
New Relic	SaaS-based, agent-based	APM, Analytics	Strong APM capabilities, user-friendly	Pricing complexity, limited customization

Key Insights

- Prometheus excels in **cloud-native** and **containerized** environments
- Open-source solutions offer more **flexibility** and **control**
- Commercial tools provide **easier setup** but at higher cost

Conclusion and Q&A

Key Takeaways

- Prometheus is a **powerful** time series database with dimensional data model
- Pull-based monitoring with **flexible** service discovery
- PromQL enables **powerful querying** and alerting
- Integrates seamlessly with **Grafana** for visualization
- Ideal for **cloud-native** and containerized environments

Resources for Further Learning

- **Official Documentation**
- **Prometheus Training**
- **Community Resources**
- **YouTube Channel**
- **GitHub Repository**
- **PromCon Conference**

Prometheus

Thank You!

Questions?